

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL
CARRERA DE SOFTWARE

PRÁCTICA EXPERIMENTAL #1

TEMA:

Fundamentos de la Ingeniería de Requisitos y Proceso de Requisitos

MATERIA:

Ingeniería de Requerimientos

DOCENTE:

Guerrero Ulloa Gleiston Ciceron

ESTUDIANTES:

Chavarria Cuenca Tahiny Mel
Muñoz Quiñonez Yeranick Esther
Valarezo Flores Steven Alexander
Valdiviezo Yun On Adamo Eduardo

CURSO:

4to “B”

QUEVEDO – LOS RIOS – ECUADOR
2026-2027 PPA

ÍNDICE

SECCIÓN 0. CONFORMACIÓN DEL EQUIPO Y SELECCIÓN DEL SISTEMA.....	4
0.1. Roles y Responsabilidades	4
0.2. Datos de los Integrantes	4
0.3. Descripción del sistema PFC.....	4
0.4. Identificación y clasificación de stakeholders.....	5
0.5. Tipo de sistema del PFC.....	5
SECCIÓN 2. DESARROLLO DEL FUNDAMENTO TEÓRICO	6
1.1. Marco normativo e integración de normas para la IR.....	6
1.2. Definición y naturaleza del requisito de software	7
1.2.1. Definición formal de requisito según IEEE/ISO/IEC 29148:2018.....	7
1.2.2. Diferencia entre necesidad del Stakeholder vs. Requisito vs. Restricción	8
1.2.3. Propiedades Emergentes del Software.....	9
1.2.4. Requisitos Cuantificables.....	9
1.2.5. Requisitos del Sistema vs. Requisitos del Software	10
1.2.6. Restricciones de Diseño, Proceso, Interfaz y Legales	10
1.3. Taxonomía completa de Tipos de Requisitos (IEEE/ISO/IEC 29148:2018).....	11
1.4. Proceso de Ingeniería de Requisitos.....	13
1.4.1. Modelos de Proceso	13
1.4.2. Actores según IEEE 29148 y PMBoK 7.a ed.	14
1.4.3. Administración y Soporte	15
1.4.4. Interrelación con el ciclo de vida del software	16
1.4.5. Perfeccionamiento: métricas de calidad del proceso, niveles CMMI, mejora continua	
16	
1.4.6. Framework de Ingeniería de Requisitos según Pohl.....	17

1.5.	Contexto, Tipos de Sistemas y Visión	19
1.5.1.	Tipos de Sistemas	19
1.5.2.	Límite del sistema según IEEE 29148: entidades internas vs. externas, interfaces	21
1.5.3.	Contexto de la IR: objeto de contexto, información de contexto, perspectiva de contexto	21
1.5.4.	Visión del Sistema: Declaración de Visión en formato Moore	22
1.5.5.	Las ocho perspectivas de contexto: Uso, Sujeto, TI, Desarrollo, Negocio, Innovación, Sociedad y Técnica.	22
1.6.	Propiedades de un Buen Requisito	24
1.6.1.	Nueve propiedades individuales según IEEE 29148:2018	24
1.6.2.	Propiedades del conjunto: completitud, consistencia, no redundancia, trazabilidad cruzada	26
1.6.3.	Diferencia entre verificación y validación en el contexto de la calidad del requisito	26
	SECCIÓN 3. ARTEFACTOS DEL PROCEDIMIENTO	27
3.1.	Definir el límite del sistema del PFC	27
3.2.	Redactar la declaración de visión en formato Moore.....	27
3.3.	Aplicar las ocho perspectivas de contexto al PFC	28
3.4.	Elaborar el mapa conceptual integrador.....	30
	CONCLUSIONES	31
	REFERENCIAS IEEE	32
	ANEXOS	35
	Anexo A. Exportar la lista de referencias IEEE	35
	Anexo B. Capturas de pantalla en Mendeley.....	35

SECCIÓN 0. CONFORMACIÓN DEL EQUIPO Y SELECCIÓN DEL SISTEMA

0.1. Roles y Responsabilidades

Rol	Integrante	Responsabilidad
Analista Líder Documentador	Muñoz Quiñonez Yeranick Esther	Coordina, revisa coherencia y citas Investiga la sección 2.1 y 2.3 Realiza la maquetación del archivo
Investigador A	Chavarria Cuenca Tahiny Mel	Investiga la sección 2.2 Elabora mapa conceptual integrador
Investigador B	Valdiviezo Yun On Adamo Eduardo	Investiga la sección 2.4 y 2.5
Investigador C	Valarezo Flores Steven Alexander	Investiga la sección 2.5 y 2.6 Redacta la declaración de visión

0.2. Datos de los Integrantes

Nombres Completos	Cédula	Correo Institucional
Chavarria Cuenca Tahiny Mel	0943050054	tchavarriac@uteq.edu.ec
Muñoz Quiñonez Yeranick Esther	1207929645	ymunozq@uteq.edu.ec
Valarezo Flores Steven Alexander	1250243241	svalarezof@uteq.edu.ec
Valdiviezo Yun On Adamo Eduardo	1250520515	avaldiviezoy@uteq.edu.ec

0.3. Descripción del sistema PFC

El Sistema de Gestión Inteligente de Aulas busca optimizar el uso de los recursos físicos en la facultad. Actualmente, la climatización, seguridad y mantenimiento se gestionan manualmente, lo que provoca consumo energético excesivo, falta de control sobre el estado de los equipos y costos operativos elevados, afectando la calidad del aprendizaje. Para resolver esta problemática, el sistema integra monitoreo IoT (sensores de temperatura, puertas, proyectores e Internet), visión artificial para identificar residuos u objetos fuera de lugar y un control remoto accesible desde una interfaz web. También ofrece un dashboard por roles, garantizando seguridad, baja latencia y escalabilidad en todas las aulas. Sus principales usuarios son las autoridades, que buscan reducir costos; los conserjes, que necesitan optimizar sus rondas de limpieza y apagado; y los estudiantes

y docentes, que requieren confort y equipos funcionales. En el contexto organizacional, el sistema impulsa la eficiencia operativa y mejora la experiencia académica.

0.4. Identificación y clasificación de stakeholders

Stakeholder	Poder	Interés	Clasificación	Justificación
Autoridades de la facultad	Alto	Alto	Key Players	Aprueban presupuesto y buscan reducir costos operativos y energéticos.
Administrador TI	Alto	Alto	Key Players	Supervisa el funcionamiento técnico del sistema y la infraestructura.
Docentes	Bajo	Alto	Keep Informed	Necesitan aulas funcionales, internet estable y proyectores operativos.
Conserjes	Bajo	Alto	Keep Informed	Utilizan el sistema para monitoreo, limpieza y control de dispositivos.
Estudiantes	Bajo	Alto	Keep Informed	Son beneficiarios directos del confort térmico y del correcto funcionamiento de las aulas.

0.5. Tipo de sistema del PFC

El sistema propuesto corresponde a un software-intensivo con características de sistema embebido, adaptativo y System of Systems (SoS). Se considera software-intensivo porque gran parte de su funcionalidad depende del procesamiento de software, visualización de datos, gestión de usuarios y control remoto mediante una plataforma web. Además, posee características de sistema embebido debido a la integración de sensores IoT y dispositivos físicos para el monitoreo de temperatura, estado de proyectores, puertas y conectividad de internet. Es también adaptativo, ya que detecta anomalías mediante visión artificial y responde automáticamente a condiciones del entorno. Por último, se clasifica como un System of Systems debido a la integración de múltiples subsistemas independientes, como sensores, módulos de IA, red institucional y dashboard administrativo que trabajan de manera coordinada.

SECCIÓN 2. DESARROLLO DEL FUNDAMENTO TEÓRICO

1.1. Marco normativo e integración de normas para la IR

A continuación, se presenta una síntesis de las principales normas y guías utilizadas en Ingeniería de Requisitos, destacando su alcance e integración con la norma IEEE/ISO/IEC 29148:2018 como eje central del proceso de especificación y gestión de requisitos de software y sistemas.

Norma/Guía	Alcance relevante para IR	Integración con IEEE 29148
IEEE/ISO/IEC 29148:2018 [1]	Norma principal de Ingeniería de Requisitos. Define procesos de IR, atributos de calidad de requisitos, documentación SRS y trazabilidad. Estudios recientes destacan que es el estándar central para formalizar requisitos y mejorar su calidad y consistencia [2], [3].	Actúa como núcleo integrador de la IR y complementa los procesos de ciclo de vida definidos por de vida definidos por IEEE/ISO/IEC 15288:2023 e IEEE/ISO/IEC 12207:2017 [4], [5].
IEEE/ISO/IEC 15288:2023 [5]	Define procesos del ciclo de vida del sistema, técnicos, organizacionales y de acuerdo. Establece la relación entre stakeholders, necesidades y requisitos del sistema [5]. Investigaciones recientes resaltan su importancia en la trazabilidad y modelado de requisitos complejos [6].	IEEE 29148 amplía y operacionaliza los procesos de requisitos descritos en 15288, especialmente en la transformación de necesidades de stakeholders en requisitos verificables y trazables [1], [5].
IEEE/ISO/IEC 12207:2017 [4]	Establece los procesos del ciclo de vida del software, incluyendo desarrollo, validación, mantenimiento y gestión de configuración [4].	IEEE 29148 complementa 12207 especificando la documentación y gestión de los requisitos de software dentro del ciclo de vida definido por dicha norma [1], [4].
ISO/IEC 25010:2023 [7]	Proporciona el modelo de calidad para sistemas y software. Estudios	IEEE 29148 utiliza las características de calidad de ISO

	destacan su relevancia para evaluar completitud, consistencia y verificabilidad [7], [8].	25010 como base para la definición y clasificación de requisitos no funcionales dentro del SRS [1], [7].
SWEBOK v4.0 [9]	Guía del conocimiento en Ingeniería de Software que aborda buenas prácticas de documentación, validación y gestión de requisitos [9].	Refuerza académicamente los procesos y actividades definidos por IEEE 29148, sirviendo como referencia complementaria para la aplicación práctica de la IR [1], [9].
PMBok 7. ^a ed. [10]	Estándar de gestión de proyectos que aborda gestión de stakeholders, alcance, planificación y control de cambios. Su enfoque ayuda a integrar la Ingeniería de Requisitos con la administración del proyecto [10].	Complementa IEEE 29148 en aspectos de gestión de cambios, control de alcance y comunicación con stakeholders durante el ciclo de vida del proyecto [1], [10].
IREB CPRE FL v3.1 [11]	Currículo profesional de certificación en Ingeniería de Requisitos que define terminología, técnicas de especificación, validación y buenas prácticas de calidad [11].	Complementa IEEE 29148 mediante lineamientos prácticos para redacción, validación y aseguramiento de calidad de requisitos en proyectos reales [1], [11].

1.2. Definición y naturaleza del requisito de software

A continuación, se desarrollan los conceptos fundamentales relacionados con la definición y naturaleza de los requisitos de software, abordando su significado formal según las normas IEEE/ISO/IEC.

1.2.1. Definición formal de requisito según IEEE/ISO/IEC 29148:2018

Un requisito es una declaración que traduce o expresa una necesidad junto con sus restricciones y condiciones asociadas. La norma ISO/IEC/IEEE 29148:2018 establece que los requisitos pueden existir en diferentes niveles dentro de la estructura de un sistema [1]. Un requisito de software indica qué debe hacer el sistema o qué condición debe cumplir para satisfacer una necesidad del

usuario o de la organización. No debe escribirse de manera ambigua, sino de forma clara para que el equipo de desarrollo pueda implementarlo correctamente. Por ejemplo, en el PFC, un requisito de software sería: “El sistema deberá permitir monitorear en tiempo real la temperatura, el estado de las puertas, el encendido de proyectores, el funcionamiento de los aires acondicionados y la calidad de la señal de Internet en las aulas de la facultad”.

1.2.2. Diferencia entre necesidad del Stakeholder vs. Requisito vs. Restricción

Una necesidad del stakeholder representa una expectativa, problema o capacidad requerida por una persona, grupo u organización interesada en el sistema. En ingeniería de requisitos, esta necesidad debe analizarse y transformarse en requisitos verificables del sistema o del software [1]. La necesidad del stakeholder es el punto de partida del requisito, pero todavía no está escrita con suficiente detalle técnico. Generalmente se expresa como un deseo, problema o carencia que tiene el usuario final. Por eso, el analista debe convertir esa necesidad en una especificación clara que pueda ser diseñada, construida y probada. Por ejemplo, una necesidad de las autoridades sería: “Queremos reducir el consumo eléctrico provocado por proyectores y aires acondicionados que quedan encendidos sin necesidad”. A partir de esa necesidad, se puede formular un requisito: “El sistema deberá permitir al personal autorizado apagar de forma remota los aires acondicionados y proyectores desde una interfaz web”.

Un requisito es una declaración formal que expresa una necesidad y sus condiciones asociadas, pudiendo estar escrita en lenguaje natural u otra forma de representación. La norma ISO/IEC/IEEE 29148:2018 indica que un requisito debe expresar adecuadamente el contenido necesario para guiar el desarrollo del sistema [1]. El requisito es más preciso que una simple necesidad, porque ya describe una función, comportamiento o condición que el sistema debe cumplir. Su finalidad es evitar interpretaciones diferentes entre usuarios, desarrolladores y evaluadores. Mientras mejor redactado esté el requisito, menor será el riesgo de errores durante el desarrollo del software. Como ejemplo, “La plataforma deberá mostrar el estado actualizado de cada aula, incluyendo temperatura, estado de puertas, encendido de proyector, funcionamiento del aire acondicionado y calidad de la señal de Internet”.

Una restricción es una limitación o condición que afecta la forma en que un sistema o software debe desarrollarse, implementarse o utilizarse. En ISO/IEC/IEEE 29148:2018, el concepto de requisito incluye tanto la necesidad como sus restricciones y condiciones asociadas [1]. La restricción no describe necesariamente una función del sistema, sino un límite que debe respetarse.

Puede estar relacionada con tecnología, tiempo, presupuesto, seguridad, normas institucionales, interfaz o aspectos legales. Las restricciones son importantes porque condicionan las decisiones técnicas del proyecto desde el inicio. Una restricción sería: “Solo el personal autorizado, como administradores, autoridades o conserjes designados, podrá ejecutar comandos físicos sobre los dispositivos, como apagar un aire acondicionado o un proyector desde la plataforma web”.

1.2.3. Propiedades Emergentes del Software

Las propiedades emergentes son características que surgen del comportamiento conjunto de los elementos de un sistema y no únicamente de un componente aislado. La norma ISO/IEC/IEEE 15288:2023 considera el sistema como un conjunto de elementos relacionados dentro de un ciclo de vida, lo cual permite analizar propiedades que aparecen por la interacción entre dichos elementos [5]. Una propiedad emergente aparece cuando los módulos del software interactúan entre sí y generan un resultado global. Por ejemplo, la seguridad, el rendimiento, la usabilidad o la disponibilidad no dependen de una sola pantalla o función, sino de todo el sistema trabajando en conjunto. Por eso, estas propiedades deben considerarse desde la etapa de requisitos y no solo al final del desarrollo. En el PFC, la eficiencia energética sería una propiedad emergente. No depende solo de apagar un aire acondicionado, sino de la combinación entre sensores de temperatura, monitoreo del estado de los equipos, dashboard administrativo, control remoto y decisiones del personal autorizado. Cuando todos estos elementos trabajan juntos, el sistema puede ayudar a reducir el consumo eléctrico de la facultad.

1.2.4. Requisitos Cuantificables

Un requisito cuantificable es aquel que puede medirse mediante criterios objetivos, métricas o condiciones verificables. ISO/IEC/IEEE 29148:2018 señala que los requisitos deben expresarse de forma adecuada para ser comprendidos y evaluados durante el desarrollo y validación del sistema [1]. Los requisitos cuantificables ayudan a evitar frases ambiguas como “el sistema debe ser rápido” o “la aplicación debe ser fácil de usar”. En su lugar, se deben usar valores medibles, como tiempo de respuesta, porcentaje de disponibilidad, número de intentos o nivel de precisión. Esto facilita comprobar si el requisito se cumplió durante las pruebas del software. Un requisito cuantificable sería: “El sistema deberá actualizar el estado de los sensores de temperatura, puertas, proyectores, aires acondicionados e Internet en un tiempo menor a 5 segundos desde que se detecte el cambio”.

1.2.5. Requisitos del Sistema vs. Requisitos del Software

Los requisitos del sistema describen las capacidades, condiciones y restricciones del sistema completo, considerando sus elementos, procesos, usuarios, hardware, software e infraestructura. ISO/IEC/IEEE 15288:2023 establece un marco común para describir procesos del ciclo de vida de sistemas creados por el ser humano [5]. Un requisito del sistema tiene una visión más amplia que un requisito de software. No se limita únicamente a la aplicación, sino que considera todo lo necesario para que el sistema funcione en un contexto real. Puede incluir usuarios, dispositivos, conexión a internet, base de datos, políticas institucionales y procesos de uso. Un requisito del sistema sería: “El sistema deberá integrar sensores IoT, cámaras de visión artificial, dispositivos de control físico y una plataforma web para monitorear y gestionar las condiciones de las aulas de la facultad”.

Los requisitos de software especifican las funciones, comportamientos, interfaces, datos y restricciones propias del producto software. ISO/IEC/IEEE 12207:2017 proporciona procesos para definir, controlar y mejorar los procesos del ciclo de vida del software dentro de una organización o proyecto [4]. Los requisitos de software se enfocan directamente en lo que debe hacer la aplicación. Incluyen funcionalidades como registrar usuarios, iniciar sesión, guardar calificaciones, mostrar lecciones o generar reportes. También pueden incluir condiciones de calidad, como seguridad, rendimiento, usabilidad y mantenibilidad. Un requisito de software sería: “La aplicación web deberá mostrar un dashboard diferenciado por roles, donde los estudiantes y docentes puedan visualizar información general del estado del aula, mientras que los administradores puedan acceder a funciones de monitoreo, alertas y control remoto”.

1.2.6. Restricciones de Diseño, Proceso, Interfaz y Legales

Las restricciones son condiciones que limitan las decisiones de diseño, desarrollo, integración o uso del software. ISO/IEC/IEEE 29148:2018 integra las restricciones como parte de la expresión formal de un requisito, ya que estas condicionan la solución que se va a construir [1]. Las restricciones de diseño indican decisiones técnicas obligatorias, como usar una arquitectura web o una base de datos específica. Las restricciones de proceso se relacionan con la metodología de trabajo, entregables, documentación o normas que debe seguir el proyecto. Las restricciones de interfaz y legales establecen cómo debe comunicarse el sistema con los usuarios u otros sistemas, y qué obligaciones normativas deben cumplirse. Una restricción de diseño sería: “El sistema deberá funcionar como una plataforma web accesible desde computadoras o dispositivos móviles

conectados a Internet”. Una restricción de interfaz sería: “El dashboard deberá mostrar indicadores visuales claros para identificar aulas con fallas técnicas, residuos detectados o consumo energético innecesario”. Una restricción legal y de seguridad sería: “El sistema deberá proteger el acceso a los controles físicos mediante autenticación y roles, evitando que estudiantes o usuarios no autorizados apaguen equipos de la facultad”.

1.3. Taxonomía completa de Tipos de Requisitos (IEEE/ISO/IEC 29148:2018)

A continuación, se presenta la taxonomía de tipos de requisitos aplicada al Sistema Inteligente de Gestión de Aulas. Cada requisito se encuentra clasificado según IEEE/ISO/IEC 29148:2018 e incluye su identificador, tipo, descripción, prioridad MoSCoW y fuente correspondiente.

ID	Tipo	Descripción	Prioridad MoSCoW	Fuente
RN-01	Negocio	El sistema deberá reducir el consumo energético de proyectores y aires acondicionados en las aulas de la facultad mediante monitoreo y control automatizado.	Must	Autoridades
RS-01	Stakeholder	Las autoridades de la facultad requieren visualizar reportes de consumo energético y estado de los dispositivos en tiempo real.	Must	Autoridades
RU-01	Usuario	Los conserjes deberán recibir alertas sobre aulas con dispositivos encendidos fuera del horario académico.	Must	Conserjes
RF-01	Funcional	El sistema deberá monitorear la temperatura, estado de puertas, proyectores y conectividad de internet mediante sensores IoT.	Must	Equipo de desarrollo

RF-02	Funcional	El sistema deberá permitir el apagado remoto de aires acondicionados y proyectores desde una interfaz web administrativa.	Must	Autoridades
RNF-01	No Funcional	El sistema deberá actualizar el estado de los sensores en un tiempo menor a 5 segundos.	Must	Equipo de desarrollo
RI-01	Interfaz	El dashboard deberá mostrar interfaces diferenciadas según el rol del usuario (administrador, docente y estudiante).	Should	Usuarios
RR-01	Rendimiento	El sistema deberá soportar el monitoreo simultáneo de todas las aulas de la facultad sin degradación significativa del rendimiento.	Must	Autoridades
RC-01	Restricción	El sistema deberá desarrollarse utilizando tecnologías compatibles con dispositivos IoT y plataformas web institucionales existentes.	Should	Facultad
RCO-01	Conformidad	El sistema deberá cumplir con las políticas institucionales de acceso y protección de datos de la universidad.	Must	Universidad
RD-01	Derivado	A partir del monitoreo de temperatura, el sistema deberá generar alertas automáticas cuando un aula supere los límites establecidos.	Could	Equipo de desarrollo
RT-01	Transición	El sistema deberá permitir una migración gradual desde el control manual hacia el monitoreo automatizado sin interrumpir las actividades académicas.	Could	Autoridades

1.4. Proceso de Ingeniería de Requisitos

A continuación, se presenta el proceso de Ingeniería de Requisitos y su relación con el ciclo de vida del software. Se abordan los modelos de desarrollo, los actores involucrados, las actividades de gestión y soporte, así como los enfoques de mejora continua y los frameworks modernos aplicados a la elicitación, documentación y validación de requisitos.

1.4.1. Modelos de Proceso

El modelo Cascada se define como un esquema secuencial rígido, donde cada fase debe completarse antes de iniciar la siguiente y los requisitos deben estar claramente establecidos desde el inicio del proyecto [12], [13]. Este modelo trabaja de manera lineal al exigir que cada fase debe completarse antes de iniciar la siguiente y posee un alto nivel de documentación ya que cada fase requiere de registros formales. Sin embargo, su rigidez dificulta la adaptación a cambios del entorno, lo que provoca que los fallos graves se detecten únicamente en la fase final de verificación.

El Modelo Iterativo se define como un enfoque basado en ciclos cortos, donde el sistema se desarrolla mediante múltiples iteraciones hasta alcanzar el resultado esperado, incorporando requisitos y cambios en cada ciclo y validándolos con pruebas continuas [13], [14]. Este modelo funciona de manera incremental, ya que el producto no se concibe como una única entrega, sino como una evolución constante donde cada ciclo produce nuevas funcionalidades. Su principal ventaja es la gestión de riesgos gracias a la evaluación continua del sistema; sin embargo, exige una planificación compleja.

El Modelo Ágil presenta un enfoque iterativo que divide el desarrollo en pequeñas funcionalidades entregadas con regularidad, donde los requisitos se refinan continuamente en sprints y existe flexibilidad total ante los cambios [12], [15]. Su principal ventaja es la detección temprana de fallos, ya que la retroalimentación en cada iteración permite detectar problemas rápidamente con pruebas concurrentes. Por otro lado, su limitación radica en el bajo nivel de documentación, ya que se prioriza la calidad del software, ofreciendo únicamente guías mínimas y soporte al usuario. Un ejemplo concreto de aplicación en este proyecto de fin de curso consiste en analizar las restricciones de los requerimientos y evaluar qué modelo se adapta mejor a cada componente del sistema inteligente de gestión de aulas.

1.4.2. Actores según IEEE 29148 y PMBoK 7.a ed.

Según la norma IEEE/ISO/IEC 29148:2018 [1] y PMBoK 7.a edición [10], la Ingeniería de Requisitos involucra distintos actores que participan en la identificación, análisis, documentación, validación y gestión de los requisitos del sistema. Cada actor cumple responsabilidades específicas dentro del proyecto y contribuye a garantizar que el sistema satisfaga las necesidades de los stakeholders.

El analista de requisitos es el actor encargado de elicitar, analizar, documentar y validar los requisitos del sistema. Actúa como intermediario entre los stakeholders y el equipo técnico, asegurando que las necesidades del negocio sean transformadas en requisitos claros, verificables y trazables [1]. En el sistema inteligente de gestión de aulas, el analista de requisitos entrevista a autoridades, docentes y conserjes para identificar problemas relacionados con el consumo energético y fallas técnicas para posteriormente documentar los requisitos obtenidos.

El cliente es la persona, organización o entidad que solicita el desarrollo del sistema y financia el proyecto [1]. En PMBoK, el cliente es considerado un stakeholder clave debido a su influencia sobre el alcance y las decisiones del proyecto [10]. Es aquel que define las necesidades generales, expectativas y objetivos del sistema, además de aprobar los requisitos establecidos. En el PFC, las autoridades de la facultad actúan como clientes, ya que buscan reducir costos operativos relacionados con proyectores y climatización. Ellos determinan necesidades como la supervisión del consumo energético y la automatización de ciertos procesos de control.

IEEE/ISO/IEC 29148:2018 señala que los stakeholders incluyen a los usuarios finales que interactúan directamente con el sistema y cuyos objetivos y necesidades deben ser considerados durante la definición y validación de requisitos [1]. El usuario final es quien utilizará el sistema una vez implementado. Su participación es importante porque proporciona información sobre problemas reales, necesidades operativas y expectativas de uso. En el proyecto, los docentes y estudiantes son usuarios finales porque utilizarán aulas monitoreadas por el sistema inteligente. Los docentes necesitan proyectores funcionales e internet estable, mientras que los estudiantes requieren un ambiente cómodo y adecuado para el aprendizaje.

IEEE/ISO/IEC 15288:2023 menciona actividades relacionadas con la arquitectura del sistema para asegurar que los requisitos puedan implementarse mediante estructuras técnicas viables y consistentes [5]. El arquitecto de software o sistema es responsable de diseñar la estructura técnica general del sistema. Analiza aspectos como escalabilidad, integración, rendimiento y

compatibilidad para garantizar que los requisitos puedan implementarse correctamente. También evalúa restricciones técnicas y propone soluciones adecuadas para la arquitectura del sistema. En el PFC, el arquitecto define cómo se conectarán los sensores IoT, el servidor web, la base de datos y el dashboard administrativo.

La guía de los fundamentos para la dirección de proyectos establece que el gestor de proyecto es responsable de coordinar recursos, stakeholders, riesgos, cronograma y cambios para asegurar el cumplimiento de los objetivos del proyecto [10]. El gestor de proyecto supervisa la planificación y ejecución general del proyecto. Su función consiste en organizar actividades, asignar responsabilidades y controlar que el desarrollo avance según el tiempo y alcance establecidos. Además, participa en la gestión de cambios y en la comunicación entre todos los stakeholders involucrados. En el PFC, el gestor de proyecto coordina las actividades relacionadas con la implementación del sistema IoT, organiza reuniones con el equipo y verifica que los requisitos definidos por las autoridades y usuarios se cumplan dentro del cronograma establecido.

1.4.3. Administración y Soporte

La gestión de proyectos se define formalmente como la combinación de conocimientos, habilidades, herramientas y técnicas para gestionar eficientemente las actividades del proyecto y satisfacer las expectativas de los interesados. De manera complementaria, se describe como una disciplina empleada para garantizar la organización eficiente y eficaz de un proyecto desde su inicio hasta su finalización [16]. Las metodologías de gestión de proyectos funcionan como marcos de trabajo estandarizados que permiten transformar objetivos abstractos en entregables tangibles mediante una planificación rigurosa. Al implementar estos sistemas, se facilita la definición clara de roles y responsabilidades, lo que evita la duplicidad de tareas y asegura que cada miembro del equipo sepa exactamente qué se espera de él. Asimismo, el uso de estos métodos permite monitorear el progreso en tiempo real frente al presupuesto y cronograma iniciales, permitiendo realizar ajustes proactivos ante cualquier desviación detectada. En el proyecto, aplicaría la gestión de proyectos organizando el desarrollo del sistema en etapas claras como: implementación de sensores IoT, desarrollo del dashboard web y despliegue del sistema de visión artificial. Cada etapa tendría tareas específicas asignadas, junto con responsables definidos dentro del equipo. Además, se realizaría un seguimiento continuo del avance comparándolo con el cronograma, permitiendo detectar retrasos o fallas tempranas, como problemas en la transmisión de datos en tiempo real o errores en la detección de objetos.

1.4.4. Interrelación con el ciclo de vida del software

La trazabilidad de requisitos se define formalmente como la capacidad de describir y seguir la vida de un requisito tanto en dirección hacia adelante como hacia atrás a través de periodos de refinamiento e iteración continuos. Esta práctica permite establecer vínculos entre requisitos, diseño, código fuente, casos de prueba y otros artefactos del desarrollo [17]. La trazabilidad actúa como un hilo conductor que garantiza que cada necesidad del cliente esté debidamente reflejada en la arquitectura y el código del sistema. En proyectos de alta complejidad, esta disciplina es vital para realizar análisis de impacto de cambios, seleccionar pruebas de regresión y verificar el cumplimiento de normativas de seguridad. Tradicionalmente, este proceso se hacía de forma manual o mediante algoritmos simples de búsqueda de palabras, pero actualmente se utilizan técnicas de aprendizaje profundo (Deep Learning) para entender el significado semántico de los documentos y conectar conceptos relacionados, aunque no utilicen las mismas palabras exactas. En el proyecto, la trazabilidad se aplicaría conectando cada requerimiento con su implementación y validación. Por ejemplo, el requisito de que “el sistema debe actualizar los datos en menos de 5 segundos” estaría vinculado con el diseño del sistema de comunicación en tiempo real (como el uso de protocolos eficientes entre sensores IoT y el servidor), con el código que gestiona la transmisión de datos, y con pruebas específicas que midan la latencia.

1.4.5. Perfeccionamiento: métricas de calidad del proceso, niveles CMMI, mejora continua

La medición del software se define formalmente como el proceso de proporcionar una indicación cuantitativa del alcance, cantidad, dimensión, capacidad o características de un proceso o producto. Específicamente, dentro del estándar ISO 9126, la calidad del producto de software se determina mediante modelos, atributos de calidad y métricas relacionadas que se utilizan para evaluar y determinar la calidad [18]. La gestión de la calidad requiere convertir la recolección de datos brutos en métricas que actúen como indicadores para que la administración pueda controlar el proceso de desarrollo. Existen dos enfoques principales: la medición directa, que cuantifica elementos tangibles como las líneas de código o el costo, y la medición indirecta, que evalúa atributos abstractos como la facilidad de mantenimiento o la confiabilidad. El uso de marcos de trabajo internacionales como el ISO 9126 es fundamental, ya que desglosa la calidad en seis características esenciales (funcionalidad, confiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad), permitiendo una evaluación holística y técnica del sistema. En el proyecto, el perfeccionamiento se aplicaría evaluando la calidad del sistema mediante métricas concretas. Por ejemplo, se podría

medir la eficiencia verificando que los datos de los sensores IoT se actualicen en menos de 5 segundos, cumpliendo el requisito de latencia. En cuanto a la usabilidad, se podría realizar pruebas con conserjes y autoridades para comprobar que el dashboard sea fácil de entender y operar.

1.4.6. Framework de Ingeniería de Requisitos según Pohl

Se establece que la Ingeniería de Requisitos se organiza mediante un framework compuesto por actividades principales y actividades transversales que permiten identificar, documentar, validar y gestionar requisitos durante todo el ciclo de vida del sistema. El framework incluye tres actividades core: elicitación, documentación y validación; dos actividades transversales: gestión y comunicación; y tres tipos de artefactos: artefactos de requisito, de contexto y de proceso [19].

La elicitación se define como el proceso de descubrir, recopilar y comprender las necesidades, expectativas y restricciones de los stakeholders relacionadas con el sistema [19]. La elicitación es la actividad inicial de la Ingeniería de Requisitos y tiene como objetivo comprender el problema que se desea resolver. Durante esta etapa se recopila información mediante entrevistas, observación, cuestionarios o talleres con stakeholders. Una buena elicitación ayuda a reducir ambigüedades. En el PFC, la elicitación se realiza entrevistando a autoridades, docentes y conserjes para conocer problemas como el desperdicio de energía, fallas de proyectores y mala climatización en las aulas.

La documentación consiste en representar y registrar los requisitos utilizando modelos, diagramas y especificaciones que permitan su comprensión y análisis [19]. Transforma la información obtenida en requisitos claros y organizados. Esta actividad permite que todos los involucrados comprendan cómo funcionará el sistema y cuáles son sus restricciones. Además, facilita la comunicación entre stakeholders y sirve como referencia durante el desarrollo y mantenimiento del software. En el sistema inteligente, la documentación incluye requisitos funcionales como el monitoreo IoT y el control remoto, así como diagramas UML, modelos ER y tablas de trazabilidad. La validación busca comprobar que los requisitos definidos representan correctamente las necesidades reales de los stakeholders y que son adecuados para el sistema propuesto [19]. Permite detectar errores, inconsistencias o requisitos faltantes antes del desarrollo del sistema. Esta actividad ayuda a asegurar que el software construido satisfaga realmente las necesidades del usuario y del negocio. Además, reduce costos asociados a correcciones tardías en etapas avanzadas del proyecto.

En el PFC, la validación puede realizarse mostrando prototipos del dashboard a docentes y autoridades para confirmar que la información mostrada y las funciones de control cumplen sus expectativas.

La gestión de requisitos incluye actividades de planificación, control de cambios, priorización y trazabilidad de requisitos durante el ciclo de vida del proyecto [19]. Asegura que los requisitos permanezcan organizados y actualizados durante todo el proyecto. También permite controlar modificaciones, conflictos y prioridades entre stakeholders. Sin una adecuada gestión, los requisitos pueden volverse inconsistentes o perder trazabilidad. En el sistema inteligente, la gestión se aplica cuando las autoridades solicitan agregar nuevas funcionalidades, como alertas automáticas por consumo excesivo de energía.

La comunicación como la actividad que facilita el intercambio de información entre stakeholders y el equipo de desarrollo durante el proceso de Ingeniería de Requisitos [19]. Es fundamental para evitar malentendidos entre usuarios y desarrolladores. Esta actividad garantiza que todos los stakeholders comprendan los requisitos y puedan expresar necesidades o cambios de forma clara. Además, favorece la colaboración y la toma de decisiones durante el proyecto. En el proyecto, la comunicación se evidencia en reuniones entre el equipo de desarrollo, autoridades y conserjes para discutir problemas detectados y validar soluciones propuestas.

Los artefactos de requisito contienen y representan los requisitos funcionales, no funcionales y restricciones del sistema, permitiendo su documentación, análisis y trazabilidad [19]. Estos artefactos facilitan la especificación y validación de los requisitos durante el desarrollo del software. En el proyecto, se aplican mediante el documento SRS, historias de usuario y requisitos relacionados con el monitoreo IoT y el control remoto de aires acondicionados.

Los artefactos de requisito son representaciones del entorno, dominio y relaciones del sistema con actores externos y procesos organizacionales [19]. Permiten comprender el contexto operativo del sistema y las necesidades de los stakeholders. En el proyecto, se aplican mediante Diagramas UML, matriz poder/interés y modelo ER.

Los artefactos de proceso son documentos y registros utilizados para planificar, controlar y dar seguimiento al proceso de Ingeniería de Requisitos [19]. Ayudan a gestionar cambios, decisiones, trazabilidad y control del proyecto. En el proyecto, se aplican mediante una matriz de trazabilidad, actas de reunión, registro de cambios y plan de Ingeniería de Requisitos.

1.5. Contexto, Tipos de Sistemas y Visión

A continuación, se analizan los distintos tipos de sistemas y su impacto en la Ingeniería de Requisitos, considerando el contexto, límites e interacciones del sistema con su entorno.

1.5.1. Tipos de Sistemas

A continuación, se analizan cinco tipos principales de sistemas y sus consecuencias directas para la IR, con aplicación al Sistema Inteligente de Gestión de Aulas.

Los sistemas de información se centran en la gestión, almacenamiento, procesamiento y presentación de datos para apoyar la toma de decisiones organizacionales. Su foco principal es la información y los flujos de datos dentro de una organización o entre organizaciones [1]. En este tipo de sistemas, los requisitos funcionales suelen describir operaciones CRUD (crear, leer, actualizar y eliminar datos), roles de usuario, permisos y reglas de negocio. Los requisitos no funcionales más relevantes son la usabilidad, la disponibilidad y la seguridad de los datos. Para la IR, esto implica que la elicitación se concentra en comprender los procesos del negocio y las necesidades de los usuarios finales. En el Sistema Inteligente de Gestión de Aulas, el componente dashboard administrativo es un SI, ya que gestiona información sobre el estado de los dispositivos, roles de usuario y reportes de consumo energético. Los requisitos de este componente incluyen la autenticación por roles, el registro histórico de eventos y la generación de alertas administrativas. Los sistemas embebidos son sistemas de software que interactúan directamente con el mundo físico a través de sensores y actuadores, operando sobre hardware con recursos limitados y bajo restricciones estrictas de tiempo real [20]. Khalil y Kurdi [20] señalan que los sistemas embebidos integran una gran cantidad de requisitos funcionales y no funcionales como tiempo real, seguridad y confiabilidad, y que más del 50 % de los errores en sistemas embebidos ocurren por una comprensión incorrecta de los requisitos. Para la IR, esto implica que deben describirse explícitamente los requisitos de tiempo real (latencias, periodos de ejecución), los requisitos de hardware (sensores, actuadores, plataforma de procesamiento) y los requisitos de seguridad funcional. También es fundamental definir con precisión los límites del sistema, distinguiendo las entidades internas del software de las entidades externas físicas. En el sistema del PFC, los sensores IoT de temperatura, estado de puertas y proyectores constituyen el componente embebido. Los requisitos de este componente incluyen que cada sensor debe entregar lectura en menos de 90 ms, que el sistema debe detectar fallos de conectividad y que el protocolo de comunicación debe garantizar baja latencia.

Un sistema software-intensivo es aquel en el que el software es el principal responsable de la funcionalidad del sistema, aunque coexista con hardware, personas y otros sistemas. La norma ISO/IEC/IEEE 15288:2023 [5] considera este tipo de sistemas dentro de su alcance general, donde el software interactúa con múltiples elementos del ciclo de vida. En los sistemas software-intensivos, los requisitos abarcan tanto las funciones del software como las restricciones derivadas de la interacción con hardware, usuarios y sistemas externos. La IR debe gestionarse de manera holista, capturando requisitos del sistema, del software y de las interfaces externas de forma integrada. El Sistema Inteligente de Gestión de Aulas en su conjunto es un sistema software-intensivo, ya que su funcionalidad central (monitoreo, control remoto, alertas, dashboard) recae en el software, aunque depende de sensores físicos y conectividad de red. Los requisitos deben describir tanto el comportamiento del software como sus dependencias con la infraestructura tecnológica institucional.

Los sistemas adaptativos son capaces de modificar su comportamiento en respuesta a cambios en el entorno o en las condiciones de operación, frecuentemente mediante mecanismos de aprendizaje automático, lógica de control o reglas de inferencia. El SWEBOK v4.0 [9] reconoce la creciente relevancia de estos sistemas en proyectos de software modernos. La IR en sistemas adaptativos debe capturar no solo el comportamiento esperado en condiciones normales, sino también los escenarios de adaptación: qué condiciones disparan el cambio de comportamiento, cuáles son los límites de la adaptación y cómo se valida que la adaptación fue correcta. Los requisitos deben describir condiciones de activación, respuestas esperadas y restricciones de seguridad sobre los cambios automáticos. En el sistema del PFC, el módulo de visión artificial es adaptativo: detecta residuos u objetos fuera de lugar y genera alertas de manera autónoma. Un requisito de este componente es que “el sistema deberá detectar la presencia de residuos en aulas con una precisión mayor al 85 % y notificar al conserje responsable en menos de 10 segundos”, definiendo tanto la condición de activación como el umbral métrico verificable.

Un System of Systems (SoS) es una configuración de sistemas independientes e interoperables que se integran para proporcionar una capacidad única que ninguno de los sistemas constituyentes podría lograr por sí solo. ISO/IEC/IEEE 15288:2023 [5] abarca explícitamente este tipo de sistemas dentro del concepto de ciclo de vida de sistemas complejos. En un SoS, la IR enfrenta el reto de que cada subsistema tiene sus propios requisitos, interfaces y propietarios. Deben gestionarse los requisitos de interoperabilidad (protocolos de comunicación entre subsistemas), los

requisitos de integración (cómo se sincronizan los datos) y las propiedades emergentes del sistema global (por ejemplo, la eficiencia energética total). También es necesario definir claramente los límites de cada subsistema y sus interfaces. El Sistema Inteligente de Gestión de Aulas es un SoS porque integra múltiples subsistemas independientes: la red de sensores IoT, el motor de visión artificial, la plataforma web con dashboard y la red institucional WiFi. Cada uno puede operar de manera autónoma, pero su integración genera la capacidad de gestión inteligente de aulas. Los requisitos deben especificar las interfaces entre subsistemas (API REST, protocolos MQTT), las condiciones de sincronización y la forma en que el fallo de un subsistema afecta al resto.

1.5.2. Límite del sistema según IEEE 29148: entidades internas vs. externas, interfaces

Es la línea de demarcación que separa el sistema de interés de su entorno, definiendo el espacio operativo, las capacidades y las interfaces externas basadas en el concepto de operaciones. Se distingue entre el sistema (entidades internas bajo control del diseño) y su entorno (entidades externas que interactúan con él) [1].

El límite del sistema establece el alcance exacto de lo que se va a construir, diferenciando claramente qué procesos son responsabilidad del software y cuáles pertenecen a usuarios o sistemas ajenos. Es fundamental para identificar las interfaces externas, ya que estas son los puntos de conexión por donde fluye la información a través de la frontera del sistema. Sin un límite bien definido, el proyecto corre el riesgo de sufrir una expansión descontrolada del alcance, afectando el presupuesto y el tiempo. En este proyecto, las entidades internas son la base de datos de telemetría y el motor de Visión Artificial. Las entidades externas son el "Aire Acondicionado" y el "Conserje". La interfaz es la red WiFi y la API REST que permite al sistema enviar el comando de apagado al equipo físico.

1.5.3. Contexto de la IR: objeto de contexto, información de contexto, perspectiva de contexto

El contexto es la parte del entorno del sistema que es relevante para la definición y comprensión de sus requisitos. Se compone del objeto de contexto (elementos del entorno), información de contexto (datos sobre esos objetos) y perspectiva de contexto (punto de vista desde el cual se observa el entorno) [21], [22].

El contexto de ingeniería de requisitos implica entender que el software no funciona en el vacío, sino rodeado de personas, procesos y otros sistemas. Identificar los objetos de contexto permite saber qué elementos influyen en el sistema, mientras que la información de contexto provee las reglas de negocio necesarias. Finalmente, las perspectivas ayudan a que ningún requisito

importante quede fuera al analizar el entorno desde diferentes ángulos. El objeto de contexto es el aula de clase; la información de contexto incluye el horario de clases y la temperatura exterior; la perspectiva de contexto es la de la Autoridad, que prioriza el ahorro de energía sobre el lujo del enfriamiento excesivo.

1.5.4. Visión del Sistema: Declaración de Visión en formato Moore

Es una declaración estratégica que describe el propósito y valor de un producto. Sigue la estructura: Para (cliente), que (necesidad), el (producto) es un (categoría) que (beneficio), a diferencia de (competencia), nuestro producto (diferenciador) [23], [24].

Esta declaración alinea las expectativas de todos los involucrados en una sola frase potente. Ayuda a identificar claramente quién es su usuario y cuál es el valor diferencial real de su propuesta frente a soluciones existentes. Es una herramienta de validación de alto nivel que justifica la existencia misma del sistema.

Ejemplo PFC:

- **Para** autoridades y conserjes de la facultad que sufren por el uso ineficiente de energía y fallas técnicas,
- **el** Sistema Inteligente de Gestión de Aulas
- **es una** plataforma de gestión automatizada de infraestructura
- **que** optimiza el consumo energético y el mantenimiento preventivo mediante IA,
- **a diferencia de** la gestión manual actual,
- **nuestro producto** integra visión artificial para detectar residuos y control remoto centralizado en tiempo real.

1.5.5. Las ocho perspectivas de contexto: Uso, Sujeto, TI, Desarrollo, Negocio, Innovación, Sociedad y Técnica.

Las perspectivas de contexto son enfoques o entes analíticos que permiten identificar factores contextuales específicos (uso, TI, negocio, etc.) que moldean los beneficios y limitaciones de una tecnología [25], [26].

Perspectiva de contexto	Definición	Pregunta clave	Ejemplo de requisito emergente
Uso	Analiza cómo los usuarios interactúan con el sistema y cómo este apoya sus actividades diarias [19].	¿Cómo el conserje apagará el aire?	El dashboard debe incluir un botón para apagar todos los aires acondicionados de un piso.
Sujeto	Se enfoca en los objetos, entidades o información que el sistema representa o procesa [19].	¿Qué datos del proyector guardamos?	El sistema debe registrar las horas de uso y estado de funcionamiento de los proyectores.
TI	Considera la infraestructura tecnológica, redes, hardware y plataformas necesarias para el funcionamiento del sistema [19].	¿Qué red usará el IoT?	Los sensores IoT deben operar sobre la red WiFi institucional en la banda de 2.4 GHz.
Desarrollo	Relacionada con el proceso de construcción, pruebas y mantenimiento del sistema [19].	¿Cómo se probará la IA?	El sistema debe utilizar un dataset local de imágenes de aulas para entrenar el modelo de detección de residuos.
Negocio	Analiza objetivos organizacionales, beneficios económicos y metas institucionales [19].	¿Cuánto dinero se ahorrará?	El sistema debe reducir el consumo eléctrico mensual de la facultad en al menos un 15%.
Innovación	Examina capacidades novedosas o disruptivas que aporten valor adicional al sistema [19].	¿Qué hace la IA además de monitorear?	El sistema debe predecir posibles fallas en los aires acondicionados mediante análisis de vibración y uso.

Sociedad	Considera aspectos éticos, legales, privacidad e impacto social del sistema [19].	¿Grabamos a los alumnos?	La visión artificial solo procesará siluetas y no almacenará rostros para proteger la privacidad de los estudiantes.
Técnica	Se enfoca en restricciones físicas, precisión, rendimiento y características técnicas del sistema [19].	¿Cuál es el rango del sensor?	El sensor de temperatura debe tener una precisión mínima de ± 0.5 °C.

1.6.Propiedades de un Buen Requisito

A continuación, se presentan las propiedades que debe cumplir un requisito para considerarse de calidad según IEEE/ISO/IEC 29148:2018.

1.6.1. Nueve propiedades individuales según IEEE 29148:2018

Son las características de calidad que debe poseer cada requisito individual para ser considerado bien formado, incluyendo ser necesario, libre de implementación, inequívoco y verificable [1], [27].

Propiedad	Definición	Ejemplo que viola la propiedad	Técnica de detección
Necesario	Es esencial para satisfacer una necesidad, objetivo o problema del sistema.	El sensor debe ser de color verde.	Verificar la trazabilidad del requisito hacia objetivos como ahorro energético o monitoreo institucional.
Libre de implementación	Describir qué debe hacer el sistema y no cómo será implementado.	El sistema usará una Raspberry Pi 4.	Identificación de tecnologías, hardware o marcas específicas dentro del requisito.

Sin ambigüedad	El requisito debe tener una única interpretación posible.	El sistema debe actualizarse rápido.	Revisión técnica utilizando métricas cuantificables, por ejemplo “menos de 5 segundos”.
Consistente	El requisito no debe entrar en conflicto con otros requisitos del sistema.	R1: El sistema apagará todos los proyectores a las 10 PM. R2: Los proyectores deben permanecer encendidos hasta las 11 PM para clases nocturnas.	Uso de matrices de conflicto y revisión cruzada de requisitos.
Completo	El requisito debe describir completamente una funcionalidad o necesidad específica sin dejar información incompleta.	El sistema monitorea temperatura, proyectores y otras cosas.	Análisis de lenguaje vago o conectores ambiguos como “etc.”, “además” u “otras cosas”.
Factible	El requisito debe poder implementarse técnica y económicamente dentro de las capacidades del proyecto.	El sistema debe predecir fallas con 100% de exactitud.	Evaluación de viabilidad técnica mediante prototipos y análisis de recursos disponibles.
Trazable	El requisito debe tener un origen identificado y relacionarse con otros artefactos del sistema.	El sistema tendrá modo invitado, sin stakeholder ni necesidad asociada.	Uso de IDs únicos y Matriz de Trazabilidad (RTM).

Verificable	El requisito debe poder comprobarse mediante pruebas, inspecciones o métricas objetivas.	El dashboard debe ser amigable.	Sustitución de términos subjetivos por criterios medibles y pruebas de usuario.
Asequible y acotado	El requisito debe mantenerse dentro de los límites de presupuesto, tiempo y alcance del proyecto.	El sistema debe implementarse en todas las facultades del país.	Análisis de alcance, recursos y tiempo disponible para el PFC.

1.6.2. Propiedades del conjunto: completitud, consistencia, no redundancia, trazabilidad cruzada

Las propiedades que debe tener el conjunto de requisitos son: Completitud (incluye todo lo necesario), Consistencia (sin contradicciones internas), No redundancia (cada requisito se dice una vez) y Trazabilidad cruzada (relación entre ellos y otros elementos) [1], [28].

Evaluar el conjunto es ver el bosque y no solo los árboles; un requisito puede estar perfecto solo, pero arruinar el grupo si contradice a otro. La no redundancia evita que el desarrollador trabaje doble o se confunda al actualizar el documento. Un conjunto de calidad asegura que no falte funcionalidad crítica para el cliente al final del desarrollo. Por ejemplo, en el proyecto la completitud se verifica si, además del monitoreo, existen requisitos para el mantenimiento de los propios sensores. La no redundancia asegura que el requisito de Seguridad no se repita en la sección de Control Remoto y en la de Visualización.

1.6.3. Diferencia entre verificación y validación en el contexto de la calidad del requisito

La verificación consiste en comprobar que las especificaciones de software estén correctamente estructuradas y cumplan de forma estricta con las reglas de calidad establecidas. Por otro lado, la validación es el proceso de confirmar que dichos requisitos definan con precisión el sistema que los stakeholders realmente necesitan para resolver su problemática [1], [21]. La verificación es técnica y formal; revisa gramática, ambigüedad y formato según el estándar IEEE. La validación es semántica y humana; se sienta con el cliente para asegurar que lo escrito realmente resolverá su problema de negocio. Un requisito puede pasar la verificación, pero fallar la validación. En el proyecto, la verificación ocurre cuando pasamos una lista de chequeo a los requisitos funcionales

para ver si son atómicos. La validación ocurre cuando el decano de la facultad confirma que los reportes de ahorro energético del sistema son exactamente lo que él necesita para el presupuesto.

SECCIÓN 3. ARTEFACTOS DEL PROCEDIMIENTO

3.1. Definir el límite del sistema del PFC

El límite del sistema permite identificar qué componentes pertenecen al sistema y cuáles interactúan desde el entorno externo. Según IEEE/ISO/IEC 29148:2018 [1], las entidades externas son actores, dispositivos o sistemas que intercambian información con el sistema mediante interfaces definidas.

En el Sistema Inteligente de Gestión de Aulas, las entidades externas corresponden a usuarios, sensores y servicios tecnológicos que interactúan con la plataforma de monitoreo y control.

Entidad externa	Tipo de datos intercambiados	Protocolo / Medio	Frecuencia
Autoridades de la facultad	Reportes, métricas, comandos administrativos	HTTPS	Bajo demanda
Conserjes	Alertas, comandos de control remoto	HTTPS	Varias veces al día
Docentes	Estado de aulas y proyectores	HTTPS	Durante horarios de clase
Red institucional	Datos de conectividad y acceso a internet	TCP/IP	Monitoreo continuo
Sistema eléctrico de las aulas	Señales de encendido/apagado de dispositivos	IoT / Relés inteligentes	En tiempo real
Servicio de autenticación institucional	Credenciales y validación de usuarios	HTTPS / API REST	En cada inicio de sesión

3.2. Redactar la declaración de visión en formato Moore

Para autoridades y conserjes de la facultad **que** sufren por el uso ineficiente de energía y fallas técnicas, **el** sistema inteligente de gestión de aulas **es una** plataforma de gestión automatizada de infraestructura **que** optimiza el consumo energético y el mantenimiento preventivo mediante IA,

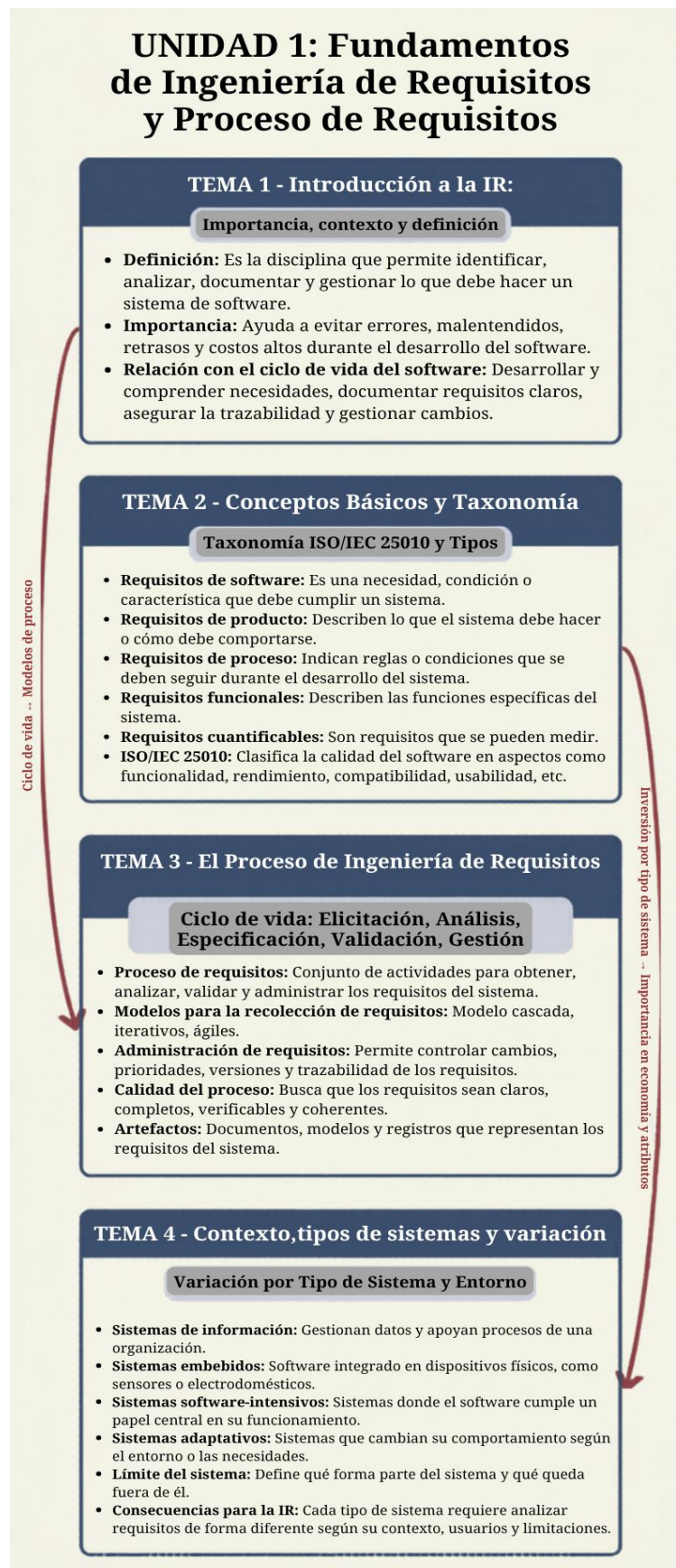
a diferencia de la gestión manual actual, **nuestro producto** integra visión artificial para detectar residuos y control remoto centralizado en tiempo real.

3.3. Aplicar las ocho perspectivas de contexto al PFC

Perspectiva	3 preguntas clave	2 requisitos/restricciones emergentes
Uso	1. ¿Cómo interactuarán los conserjes con el sistema? 2. ¿Qué acciones podrán realizar las autoridades desde el dashboard? 3. ¿Cómo recibirán alertas los usuarios?	El dashboard debe permitir apagar remotamente los aires acondicionados por piso. El sistema debe enviar alertas visuales cuando se detecten fallas en proyectores o internet.
Sujeto	1. ¿Qué información se almacenará de los proyectores? 2. ¿Qué datos ambientales serán monitoreados? 3. ¿Qué eventos deben registrarse históricamente?	El sistema debe registrar las horas de uso de cada proyector. El sistema debe almacenar el historial de temperatura de las aulas durante al menos 30 días.
TI	1. ¿Qué infraestructura de red utilizarán los sensores IoT? 2. ¿Cómo se comunicará el sistema con los dispositivos? 3. ¿Qué tecnologías soportarán el monitoreo en tiempo real?	Los sensores IoT deben operar sobre la red WiFi institucional de 2.4 GHz. El sistema debe actualizar el estado de los dispositivos en menos de 5 segundos.
Desarrollo	1. ¿Cómo se entrenará el modelo de visión artificial? 2. ¿Cómo se probarán los sensores y dispositivos? 3. ¿Qué metodología de desarrollo se utilizará?	El sistema debe utilizar un dataset local de imágenes de aulas para entrenar la IA. Cada módulo IoT debe ser validado mediante pruebas individuales antes de integrarse al sistema.

Negocio	1. ¿Qué costos operativos se desean reducir?	El sistema debe reducir el consumo eléctrico mensual en al menos un 15%.
	2. ¿Qué beneficios espera obtener la facultad?	El sistema debe generar reportes mensuales de consumo energético por aula.
	3. ¿Cómo se medirá el ahorro energético?	
Innovación	1. ¿Qué capacidades inteligentes tendrá el sistema?	El sistema debe detectar residuos u objetos fuera de lugar mediante visión artificial.
	2. ¿Cómo puede la IA anticipar problemas?	El sistema debe predecir posibles fallas en aires acondicionados mediante patrones de uso.
	3. ¿Qué funciones diferenciarán al sistema de una gestión manual?	
Sociedad	1. ¿Cómo se protegerá la privacidad de los estudiantes?	El sistema no debe almacenar rostros de estudiantes ni docentes.
	2. ¿Qué regulaciones deben respetarse?	Solo personal autorizado podrá acceder al módulo de control físico de dispositivos.
	3. ¿Cómo evitar el uso indebido de los datos recolectados?	
Técnica	1. ¿Cuál será la precisión mínima de los sensores?	Los sensores de temperatura deben tener una precisión mínima de ± 0.5 °C.
	2. ¿Qué disponibilidad debe tener el sistema?	El sistema debe soportar el monitoreo simultáneo de todas las aulas de la facultad.
	3. ¿Qué límites de rendimiento tendrá la plataforma?	

3.4. Elaborar el mapa conceptual integrador



CONCLUSIONES

El análisis de requisitos permite transformar una problemática institucional en especificaciones claras, verificables y útiles para el desarrollo del sistema. Según ISO/IEC/IEEE 29148:2018 [1], un requisito debe expresar una necesidad junto con sus condiciones y restricciones asociadas. Por ello, en el PFC, definir correctamente los requisitos sobre monitoreo IoT, control remoto, seguridad y latencia facilita construir una solución tecnológica orientada a optimizar la infraestructura física de la facultad.

Se logró comprender que la Ingeniería de Requisitos no se limita únicamente a redactar requisitos, sino que involucra procesos de análisis, validación, gestión y comprensión del contexto del sistema. Las normas IEEE/ISO/IEC 29148, 15288 y 12207, junto con los enfoques de Pohl, evidencian que un sistema de software debe desarrollarse considerando stakeholders, restricciones, calidad, trazabilidad y contexto operativo [1], [4], [5]. En el caso del PFC, esto permitió identificar actores, requisitos emergentes, límites del sistema y perspectivas de contexto que influyen directamente en la solución propuesta.

Se consiguió identificar, analizar y documentar correctamente las necesidades de los stakeholders antes de iniciar la implementación del sistema, comprendiendo que esto constituye una etapa fundamental en el desarrollo de software. Esto se evidencia en las normas IEEE/ISO/IEC 29148:2018 [1] y en el framework propuesto por Pohl [19]. Además, este conocimiento resulta esencial para la formación profesional en Ingeniería de Software, debido a que una adecuada Ingeniería de Requisitos reduce errores, mejora la comunicación entre stakeholders y aumenta las probabilidades de éxito del proyecto.

La Ingeniería de Requisitos y un ciclo de vida bien definido es esencial para el éxito de cualquier desarrollo de software [1], [5]. Estas metodologías permiten gestionar roles adecuadamente, mitigar riesgos y asegurar la trazabilidad de cada necesidad del cliente [1], [19]. Al considerar el proyecto como un Sistema de Sistemas, se logra una integración tecnológica que optimiza recursos de manera inteligente y mejora la interoperabilidad entre componentes [5]. En conclusión, la Ingeniería de Requisitos y el ciclo de vida del desarrollo de software constituyen pilares fundamentales para garantizar productos de alta calidad, mantenibilidad y rentabilidad [4], [9].

REFERENCIAS IEEE

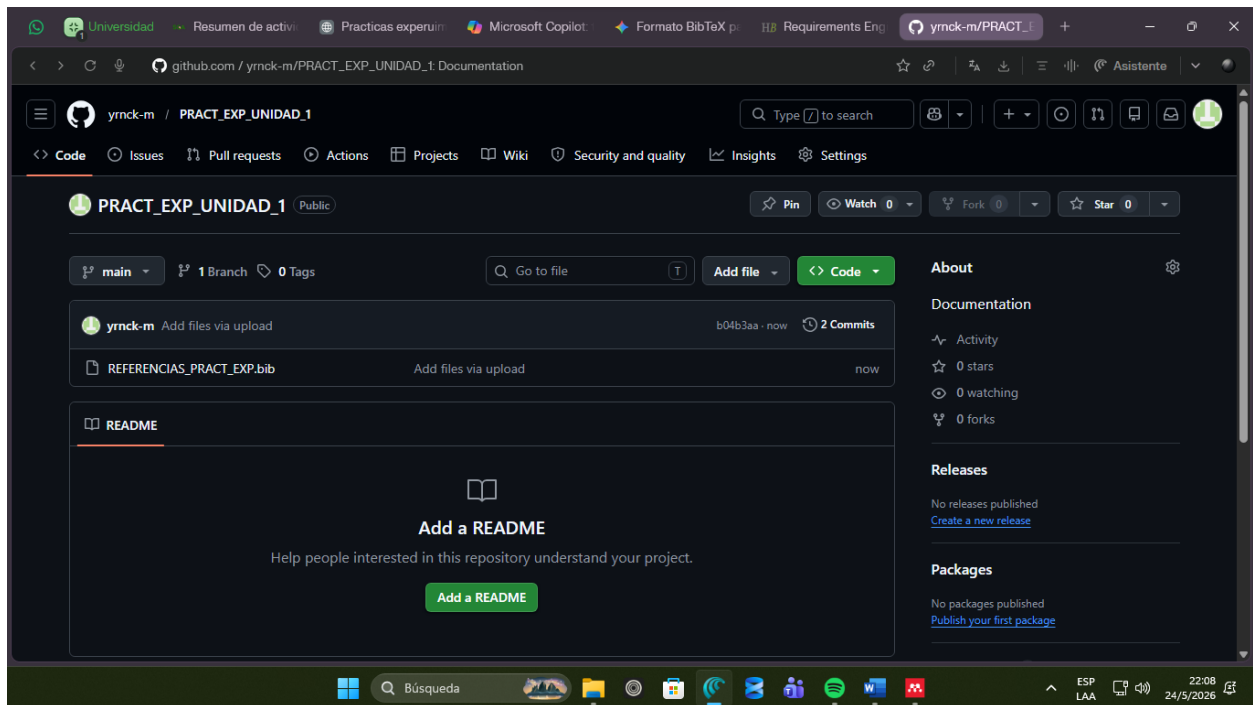
- [1] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018 – Systems and software engineering – Life cycle processes – Requirements engineering,” Geneva, Switzerland, 2018. Accessed: May 14, 2026. [Online]. Available: <https://www.iso.org/es/contents/data/standard/07/20/72089.html>
- [2] X. Franch, M. Glinz, D. Mendez, and N. Seyff, “A Study About the Knowledge and Use of Requirements Engineering Standards in Industry,” *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3310–3325, Sep. 2022, doi: 10.1109/TSE.2021.3087792.
- [3] X. Franch, C. Palomares, C. Quer, P. Chatzipetrou, and T. Gorschek, “The state-of-practice in requirements specification: an extended interview study at 12 companies,” *Requir. Eng.*, vol. 28, no. 3, pp. 377–409, Sep. 2023, doi: 10.1007/s00766-023-00399-7.
- [4] ISO/IEC/IEEE, “12207-2017 - ISO/IEC/IEEE International Standard - Systems and software engineering -- Software life cycle processes,” Sep. 28, 2017, *IEEE, Piscataway, NJ, USA*. doi: 10.1109/IEEESTD.2017.8100771.
- [5] ISO/IEC/IEEE, “ISO/IEC/IEEE 15288:2023 – Systems and software engineering – System life cycle processes,” Geneva, Switzerland, 2023. Accessed: May 14, 2026. [Online]. Available: <https://www.iso.org/standard/81702.html>
- [6] J. S. Wheaton and D. R. Herber, “Digital requirements engineering with an INCOSE-derived SysML meta-model,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.16330>
- [7] ISO/IEC, “ISO/IEC 25010:2023, Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—Product Quality Model,” 2023, *Geneva, Switzerland*.
- [8] K. Ahmad, M. Abdelrazek, C. Arora, M. Bano, and J. Grundy, “Requirements engineering for artificial intelligence systems: A systematic mapping study,” *Inf. Softw. Technol.*, vol. 158, p. 107176, Jun. 2023, doi: 10.1016/j.infsof.2023.107176.
- [9] IEEE, *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0*. Piscataway, NJ: IEEE Computer Society, 2024.
- [10] P. Management. Institute, *Guía de los Fundamentos para la Dirección de Proyectos (Guía del PMBOK) – Séptima edición y El Estándar para la Dirección de Proyectos*. Project Management Institute, Inc, 2021.

- [11] International Requirements Engineering Board, “CPRE Foundation Level Syllabus,” 2025. [Online]. Available: <https://cpre.ireb.org/en/downloads-and-resources/downloads>
- [12] A. Sinha and P. Das, “Agile Methodology Vs. Traditional Waterfall SDLC: A case study on Quality Assurance process in Software Industry,” in *2021 5th International Conference on Electronics, Materials Engineering and Nano-Technology, IEMENTech 2021*, Institute of Electrical and Electronics Engineers Inc., 2021. doi: 10.1109/IEMENTech53263.2021.9614779.
- [13] I. Alam, N. Sarwar, and I. Noreen, “Statistical analysis of software development models by six-pointed star framework,” *PLoS One*, vol. 17, no. 4 April, Apr. 2022, doi: 10.1371/journal.pone.0264420.
- [14] C. Wiecher, H. Anacker, and R. Dumitrescu, “Iterative and Scenario-based Requirements Specification in a System of Systems Context,” Feb. 2021. doi: https://doi.org/10.1007/978-3-030-73128-1_12.
- [15] A. Mishra and Y. I. Alzoubi, “Structured software development versus agile software development: a comparative analysis,” *International Journal of System Assurance Engineering and Management*, vol. 14, no. 4, pp. 1504–1522, Aug. 2023, doi: 10.1007/s13198-023-01958-5.
- [16] A. Simonaitis, M. Daukšys, and J. Mockienė, “A Comparison of the Project Management Methodologies PRINCE2 and PMBOK in Managing Repetitive Construction Projects,” *Buildings*, p., 2023, doi: 10.3390/buildings13071796.
- [17] J. L. C. Guo, J. Cheng, and J. Cleland-Huang, “Semantically Enhanced Software Traceability Using Deep Learning Techniques,” in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, 2017, pp. 3–14. doi: 10.1109/icse.2017.9.
- [18] D. Setiadi, T. Sumitra, A. Karim, and Ritzkal, “Software Quality Measurement Analysis on Academic Information Systems,” *Ingénierie des systèmes d information*, p., 2024, doi: 10.18280/isi.290418.
- [19] Klaus. Pohl, *Requirements engineering: fundamentals, principles, and techniques*. Springer, 2025.
- [20] M. Khalil and E. A. Kurdi, “EMBEDDED SYSTEMS REQUIREMENTS: A SYSTEMATIC REVIEW OF DESCRIPTION LANGUAGES AND INTELLIGENT SOFTWARE SYNTHESIS,” 2025.

- [21] A. M. Samir and M. M. Rahman, “Improving IR-based bug localization with semantics-driven query reduction,” *J. Syst. Softw.*, vol. 237, p. 112860, 2026, doi: 10.1016/j.jss.2026.112860.
- [22] A. Gawer, “Digital platforms’ boundaries: The interplay of firm scope, platform sides, and digital interfaces,” *Long Range Plann.*, vol. 54, no. 5, p. 102045, 2021, doi: 10.1016/j.lrp.2020.102045.
- [23] A. Koukias *et al.*, “Approach on Analysis of Heterogeneous Requirements in Software Engineering,” in *Proceedings of the International Federation of Automatic Control (IFAC)*, São Paulo, Brazil, 2013, pp. 372–377.
- [24] M. Tanhaei and R. Alizadehsani, “GEM-LLM: Identifying contextual equivalent mutants via large language models; A global invariant-based approach,” *Intelligent Systems with Applications*, vol. 30, p. 200640, 2026, doi: 10.1016/j.iswa.2026.200640.
- [25] A. Ricci, M. Viroli, and M. Cimadamore, “Prototyping Concurrent Systems with Agents and Artifacts: Framework and Core Calculus,” *Electron. Notes Theor. Comput. Sci.*, vol. 194, pp. 111–132, 2008, doi: 10.1016/j.entcs.2008.03.102.
- [26] M. De Donà, “Matching institutionalized expertise with global needs: Boundary organizations and hybrid management at the science-policy interfaces of soil and land governance,” *Environ. Sci. Policy*, vol. 123, pp. 82–90, 2021, doi: 10.1016/j.envsci.2021.05.015.
- [27] L. Buš, “Example Software Requirements Specification Document for ReqView,” Jun. 2019.
- [28] T. V. Ribeiro *et al.*, “Design, execution, and contextual factors shaping the benefits and drawbacks of focus groups in software engineering,” *Inf. Softw. Technol.*, vol. 195, p. 108136, 2026, doi: 10.1016/j.infsof.2026.108136.

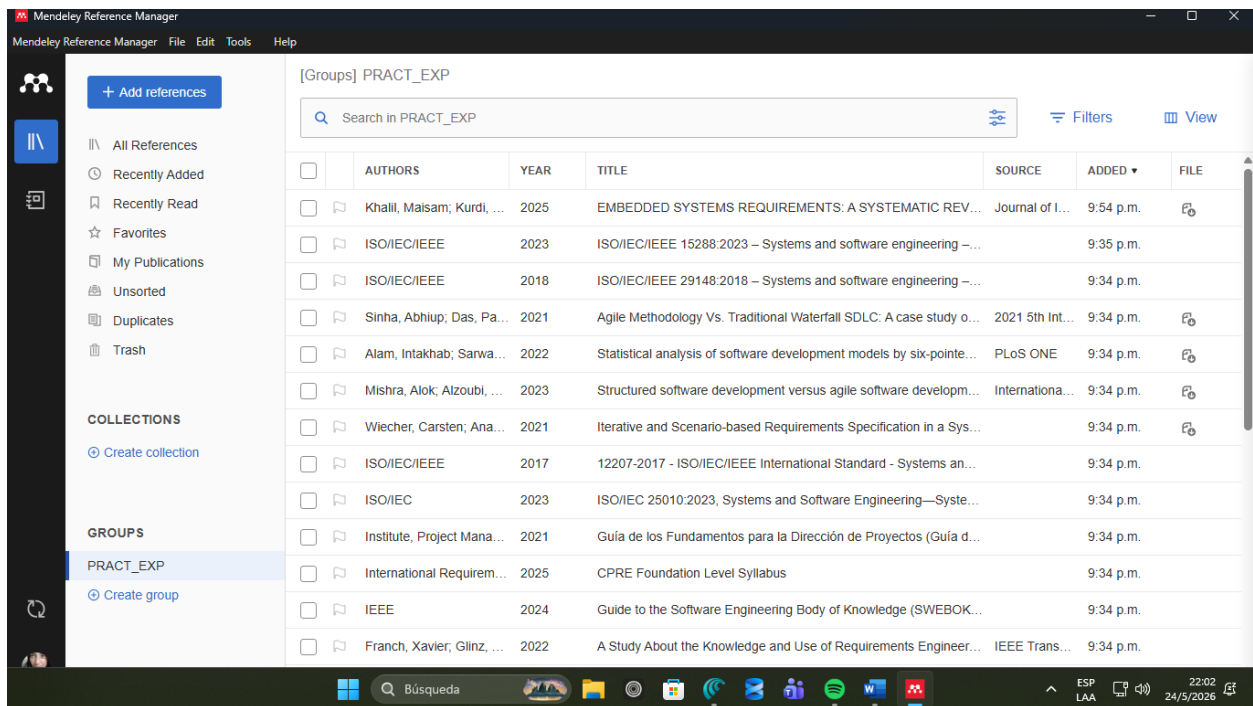
ANEXOS

Anexo A. Exportar la lista de referencias IEEE



Link de acceso: https://github.com/yrnck-m/PRACT_EXP_UNIDAD_1.git

Anexo B. Capturas de pantalla en Mendeley



Mendeley Reference Manager

Mendeley Reference Manager File Edit Tools Help

+ Add references

Search in PRACT_EXP

Filters View

	AUTHORS	YEAR	TITLE	SOURCE	ADDED	FILE
☐	Franch, Xavier; Palom...	2023	The state-of-practice in requirements specification: an extended i...	Requireme...	9:34 p.m.	
☐	Wheaton, James S; He...	2024	Digital requirements engineering with an INCOSE-derived SysML ...		9:34 p.m.	
☐	Ahmad, Khlood; Abdelr...	2023	Requirements engineering for artificial intelligence systems: A sys...	Information ...	9:34 p.m.	
☐	Simonaitis, Alvaras; Da...	2023	A Comparison of the Project Management Methodologies PRINC...	Buildings	9:34 p.m.	
☐	Guo, Jin L C; Cheng, J...	2017	Semantically Enhanced Software Traceability Using Deep Learnin...	2017 IEEE/...	9:34 p.m.	
☐	Setiadi, Dedi; Sumitra, ...	2024	Software Quality Measurement Analysis on Academic Information...	Ingénierie d...	9:34 p.m.	
☐	Pohl, Klaus.	2025	Requirements engineering : fundamentals, principles, and techniq...		9:34 p.m.	
☐	Koukias, Andreas; May...	2013	Approach on Analysis of Heterogeneous Requirements in Softwar...	Proceeding...	9:34 p.m.	
☐	Ribeiro, Talita Vieira; d...	2026	Design, execution, and contextual factors shaping the benefits an...	Information ...	9:34 p.m.	
☐	Gawer, Annabelle	2021	Digital platforms' boundaries: The interplay of firm scope, platform...	Long Rang...	9:34 p.m.	
☐	Buš, Libor	2019	Example Software Requirements Specification Document for Req...		9:34 p.m.	
☐	Tanhaei, Mohammad; ...	2026	GEM-LLM: Identifying contextual equivalent mutants via large lan...	Intelligent S...	9:34 p.m.	
☐	Samir, Asif Mohammed...	2026	Improving IR-based bug localization with semantics-driven query r...	The Journal...	9:34 p.m.	

ESP LAA 22:03 24/5/2026

Mendeley Reference Manager

Mendeley Reference Manager File Edit Tools Help

+ Add references

Search in PRACT_EXP

Filters View

	AUTHORS	YEAR	TITLE	SOURCE	ADDED	FILE
☐	Ahmad, Khlood; Abdelr...	2023	Requirements engineering for artificial intelligence systems: A sys...	Information ...	9:34 p.m.	
☐	Simonaitis, Alvaras; Da...	2023	A Comparison of the Project Management Methodologies PRINC...	Buildings	9:34 p.m.	
☐	Guo, Jin L C; Cheng, J...	2017	Semantically Enhanced Software Traceability Using Deep Learnin...	2017 IEEE/...	9:34 p.m.	
☐	Setiadi, Dedi; Sumitra, ...	2024	Software Quality Measurement Analysis on Academic Information...	Ingénierie d...	9:34 p.m.	
☐	Pohl, Klaus.	2025	Requirements engineering : fundamentals, principles, and techniq...		9:34 p.m.	
☐	Koukias, Andreas; May...	2013	Approach on Analysis of Heterogeneous Requirements in Softwar...	Proceeding...	9:34 p.m.	
☐	Ribeiro, Talita Vieira; d...	2026	Design, execution, and contextual factors shaping the benefits an...	Information ...	9:34 p.m.	
☐	Gawer, Annabelle	2021	Digital platforms' boundaries: The interplay of firm scope, platform...	Long Rang...	9:34 p.m.	
☐	Buš, Libor	2019	Example Software Requirements Specification Document for Req...		9:34 p.m.	
☐	Tanhaei, Mohammad; ...	2026	GEM-LLM: Identifying contextual equivalent mutants via large lan...	Intelligent S...	9:34 p.m.	
☐	Samir, Asif Mohammed...	2026	Improving IR-based bug localization with semantics-driven query r...	The Journal...	9:34 p.m.	
☐	De Donà, Matteo	2021	Matching institutionalized expertise with global needs: Boundary o...	Environmen...	9:34 p.m.	
☐	Ricci, Alessandro; Virol...	2008	Prototyping Concurrent Systems with Agents and Artifacts: Frame...	Electronic N...	9:34 p.m.	

ESP LAA 22:03 24/5/2026